

Python : Les listes

- ▶ Une liste stocke un ensemble d'éléments dans un ordre particulier (une séquence), elles permettent de stocker un grand nombre de données de **différents types** au même endroit.
- ▶ La liste est le type de données le plus polyvalent disponible dans Python qui peut être écrit comme une liste de valeurs séparées par des virgules (éléments) entre crochets. Les éléments de la liste peuvent être modifiés.
- ▶ **Création et initialisation d'une liste :**
- ▶ On peut créer une liste vide ou initialement remplie. Python reconnaît les listes automatiquement.
- ▶ Exemples :
 - ▶ `L1 = []` # liste vide
 - ▶ `L2 = [1, 2, 3]` # liste d'entiers
 - ▶ `L3 = [1, "Hello", 3.4]` # liste contenant différents types de données

- Pour initialiser une liste, on peut ajouter les éléments au moment de la déclaration, ou l'initialiser via une boucle for .

- Méthode 1 :

```
x=[1,'bien',20.25,3,'zineb']
```

- Méthode 2 :

```
L = []  
for i in range(0,10):  
    L.append(i**2)  
print(L)
```

- Remarque : on |



```
L = [x*3 for x in 'ABCD']  
print(L)  
# Prints [AAA, 'BBB', 'CCC', 'DDD']
```

Example :

```
L = [-12, -20, 10, 15, 42]

L1 = [x for x in L if x >= 0]

print(L1)

# Prints [10,15,42]
```

```
L = [-12, -20, 10, 15, 42]
L1 = []
for x in L:
    if x >= 0:
        L1.append(x)
print(L1)
# Prints [10,15,42]
```

Accès aux éléments d'une liste

- ▶ On peut accéder à un élément via son indice (positif ou négatif). On peut aussi récupérer une plage d'éléments en utilisant l'opérateur ' : '.
- ▶ Exemples :
- ▶ - `print(x[0])` #le premier élément de la liste
- ▶ - `print(x[-1])` #le dernier élément de la liste
- ▶ - `x=[61,'142',3,'k',1,'f']`
- ▶ `print(x[0:3])` #les éléments entre l'index 0 et l'index 2
- ▶ `print(x[1:])` #les éléments après l'index 0
- ▶ `print(x[:2])` #les éléments avant l'index 2

Parcourir une liste

méthode 1 : x =element de la liste

```
for x in L:  
    print(x)
```

méthode 2 : x=indice

```
for x in range(len(L)):  
    print(notes[x])
```

On peut utiliser un ensemble de méthodes prédéfinies pour manipuler les listes : `append()`, `insert()`, `extend()`, `pop()`, `remove()`, `split()`, `join()`

- ▶ **list.append(x)** : Ajoute un élément à la fin de la liste. Équivalent à `a[len(a):] = [x]`.
- ▶ **list.extend(iterable)** : Étend la liste en y ajoutant tous les éléments de l'itérable. Équivalent à `a[len(a):] = iterable`.
- ▶ **list.insert(i, x)** : Insère un élément à la position indiquée. Le premier argument est la position de l'élément courant avant lequel l'insertion doit s'effectuer, donc `a.insert(0, x)` insère l'élément en tête de la liste et `a.insert(len(a), x)` est équivalent à `a.append(x)`.
- ▶ **list.remove(x)** : Supprime de la liste le premier élément dont la valeur est égale à x. Une exception `ValueError` est levée s'il n'existe aucun élément avec cette valeur.
- ▶ **list.pop([i])** : Enlève de la liste l'élément situé à la position indiquée et le renvoie en valeur de retour. Si aucune position n'est spécifiée, `a.pop()` enlève et renvoie le dernier élément de la liste (les crochets autour du i dans la signature de la méthode indiquent que ce paramètre est facultatif).
- ▶ **list.clear()** : Supprime tous les éléments de la liste. Équivalent à `del a[:]`.
- ▶ **list.index(x[, start[, end]])** : Renvoie la position du premier élément de la liste dont la valeur égale x. Une exception `ValueError` est levée si aucun élément n'est trouvé.
- ▶ **list.count(x)** : Renvoie le nombre d'éléments ayant la valeur x dans la liste.
- ▶ **list.sort(key=None, reverse=False)** : Ordonne les éléments dans la liste (les arguments peuvent personnaliser l'ordonnancement, voir `sorted()` pour leur explication).
- ▶ **list.reverse()** : Inverse l'ordre des éléments dans la liste.
- ▶ **list.copy()** : Renvoie une copie superficielle de la liste. Équivalent à `a[:]`.

- ▶ Exercice1 :
- ▶ nous allons lire les notes de ahmad et khalid dans n matières (coefficient =1 pour chaque matière). on veut afficher les moyennes d'ahmad et khalid puis le nom de celui ayant
- ▶ la meilleure moyenne

Exercice D'application:

► Exercice 1 :

- programme en python qui permet de sauvegarder les noms de 10 stagiaires saisi par l'utilisateur.

► Exercice 2 :

- Remplir une liste avec 10 entiers donnés par l'utilisateur .Puis, le programme trouve le plus grand et le plus petit de ces nombres.

► Exercice 3 ;

- Proposer un programme qui demande un nombre entier x entre 1 et 10 jusqu'à ce que la réponse convienne. Puis, le programme remplit une liste L avec les multiplicateurs de x qui sont entre 0 et 100.On affiche alors la liste

► Exercice 4 :

- Programme qui permet de lire 10 note. Puis affiche les notes qui sont >moyenne

Dictionnaire

Définition et Utilisation

1. Dictionnaire :

- ▶ Un dictionnaire est une collection de données non ordonnée, modifiable et n'autorisant pas les doublons. En effet, un dictionnaire permet de stocker une valeur en lui associant une clé, et fournissant un accès très rapide à la valeur à partir de la clé.
- ▶ Le dictionnaire Python fonctionne de la même manière qu'un **dictionnaire bilingue**. Par exemple, on a un mot français et on a besoin de son équivalent en arabe.
- ▶ Dans le monde de Python, le mot qu'on recherche est nommé **key** et le mot qu'on obtient du dictionnaire est appelé **value**. Cela signifie qu'un dictionnaire est un ensemble de paires **clé-valeur**.

► **Remarque :**

- Chaque clé doit être **unique**
- Une clé peut être des **données de tout type**(sauf liste) : elle peut être un nombre (entier ou flottant), voire une chaîne, un tuple ;
- Un dictionnaire n'est pas une liste : une liste contient un ensemble de valeurs numérotées, tandis qu'un **dictionnaire contient des paires de valeurs**;
- La fonction len() fonctionne également pour les dictionnaires - elle renvoie le nombre d'éléments de valeur-clé dans le dictionnaire;
- Un dictionnaire est un **outil à sens unique**: si on a un dictionnaire anglais-français, on peut rechercher des équivalents français de termes anglais, mais pas l'inverse.

► **Syntaxe :**

- Chaque dictionnaire est un ensemble de paires *clé-valeur*. On peut le créer en utilisant la syntaxe suivante:

```
myDictionary = { key1 : value1,  
                  key2 : value2,  
                  key3 : value3,...  
                }
```

► Exemple:

```
dict1 = {"cat" : "chat", "dog" : "chien", "horse" : "cheval"}  
  
dict2 = {}#dictionnaire vide  
  
print(dict1)  
  
print(dict2)
```

1. **Accès aux éléments :**

► Pour obtenir une des valeurs, on doit fournir une valeur de clé valide.

► Exemple :

► `print(dict1['cat'])` Résultats : chat

- ▶ Il existe également une méthode appelée `get ()` qui vous donnera le même résultat:
- ▶ `print(dict1.get('cat'))`
- 2. Utiliser la méthode `keys()` :
 - ▶ Cette méthode **renvoie un objet itérable composé de toutes les clés rassemblées dans le dictionnaire**. Avoir un groupe de clés nous permet d'accéder à l'ensemble du dictionnaire de manière simple et pratique.

► Exemple :

```
1 dict1 = {"cat" : "chat", "dog" : "chien", "horse" : "cheval"}
2
3 for key in dict1.keys():
4
5     print(key, "->", dict1[key])
```

► 3- Ajout d'une nouvelle clé

- L'ajout d'un élément au dictionnaire se fait en utilisant **une nouvelle clé** et en lui attribuant une valeur. en utilisant la méthode **update()**, par exemple:

```
dictionary = {"cat" : "chat", "dog" : "chien", "horse" :  
"cheval"}  
dictionary.update({"duck" : "canard"})  
print(dictionary)
```

4. Retrait d'une clé

- ▶ La suppression d'une clé entraînera toujours la **suppression de la valeur associée. Les valeurs ne peuvent exister sans leurs clés.**
- ▶ l'instruction `del`. Exemple :

```
dictionary = {"cat" : "chat", "dog" : "chien", "horse" :  
"cheval"}  
  
    del dictionary['dog']  
    print(dictionary)
```

Remarque :

- ▶ La méthode `clear ()` vide le dictionnaire : **`dict.clear()`**
- ▶ La méthode **`pop ()`** supprime l'élément avec le nom de clé spécifié:
`dictionary.pop('dog')`